CS 554 Project Check-In

Analyzing and Restoring Mutual Exclusion in Lock Algorithms Across Hardware and Language Memory Models

Krishna Riteshkumar Patel (UIN: 670178497)
CS 554, Fall 2026

1. Progress and implementation

The project studies which memory-order assumptions make classical lock entry protocols safe, and how stronger synchronization changes the allowed executions. Following the proposal feedback, the implementation reuses the existing `sc.cat` and `x86tso.cat` from herdtools7. A repeatable local runner executes herd7 7.58 and CDSChecker and saves raw logs, JSON summaries, and counterexample DOT graphs alongside the source files.

The current suite contains 13 finite litmus tests and four models, giving 46 applicable model/test runs. The custom RA and SRA encodings follow the all-release write/all-acquire read setting of Lahav et al. [3]. They are language-level abstractions; the x86-shaped test syntax is only an event generator for those runs. Store buffering, message passing, and 2+2W provide calibration tests; these do not prove equivalence to the course definitions. An independent FIFO-store-buffer checker agrees with herd7 on all observed register outcomes in eight finite SC/TSO comparisons.

2. Initial axiomatic results

The Peterson tests check the full simultaneous-admission condition, not only the case where both threads read zero flags. Let a_i be the other flag read by thread i, and t_i its read of `turn`. Each thread reads both once, unconditionally, without retry or unlock. The violation condition is

$$(a_0 = 0 \ \vee \ t_0 = 0) \ \wedge \ (a_1 = 0 \ \vee \ t_1 = 1).$$

Entry test / calibration	SC	TSO	RA	SRA
Peterson: plain stores	0	6	6	6
Peterson: MFENCE	0	0	–	–
Peterson: dummy RMW	0	0	2	2
Peterson: turn RMW	0	0	0	0
Dekker doorway	0	1	1	1
2+2W calibration	0	0	1	0

Entries are herd7 *positive execution-witness counts*, not probabilities or counts of distinct register states. Zero means the specified bad outcome was forbidden in this finite test. A dash marks a hardware fence outside the RA/SRA interpretation. The Dekker test covers its first flag-check doorway, not its complete retry/turn protocol; the 2+2W outcome is $x = 1 \wedge y = 1$.

3. A problem encountered: blocking one outcome is insufficient

The plain Peterson fragment is unsafe under TSO, RA, and SRA. An x86 `MFENCE` after each turn store removes the TSO violation. However, one acquire-release exchange on a shared dummy location after that store only removes the *both-zero* subset under RA/SRA; two violating witnesses remain for the full admission condition. One has $a_0 = 0$, $t_0 = 1$, $a_1 = 1$, $t_1 = 1$: thread 0 passes on its flag read, and thread 1 passes on its turn read. Replacing the *turn store itself* with an acquire-release exchange is a separate variant with zero violating witnesses in all four current models. This is evidence for that bounded variant, not an unbounded correctness proof.

AI usage. I used AI to organize the project files, draft README.md and edit documentation, report and debug the tool setup and scripts.

4. C/C++11 experiments with CDSChecker

The language-level suite separates an admission assertion from an instrumented non-atomic payload race check. The admission test records each thread's private decision and asserts after joining, avoiding a shared monitor RMW that could change the synchronization. Payload accesses use `store_32`/`load_32` from CDSChecker's race detector. Stores use release and loads use acquire in the release/acquire variants; `acq_rel` is reserved for suitable RMWs.

CDSChecker harness	Total	Buggy
Peterson entry: relaxed	12	5
Peterson entry: release/acquire	12	5
Peterson entry: seq_cst	8	0
Peterson payload: relaxed	104	39
Peterson payload: release/acquire	44	7
Peterson payload: seq_cst	34	0
TAS try-lock: acquire/release	6	0
Ticket lock: acquire/release	16	0
SPSC one-item publication	2	0

Total includes complete, buggy, infeasible, and redundant candidates, recorded separately in JSON. Admission bugs are assertion violations. Payload race reports identify missing happens-before ordering, not necessarily simultaneous occupancy. These results concern the historical CDSChecker C11/C++11 implementation, rather than the custom RA/SRA models.

All tests use two worker threads. Admission tests make one guard observation each; Peterson payload probes allow two and withdraw on exhaustion. TAS makes one try-lock attempt; ticket uses one ticket and at most three serving polls per thread. SPSC checks one-item publication without slot reuse. Zero-bug results concern these safety bounds, not fairness, termination, or a full queue.

5. Technical issues and remaining work

Herd represents an update with separate read and write events. The RA/SRA encodings therefore include its internal `rmw` edge in happens-before, retain per-location coherence, and check update atomicity. SRA additionally requires acyclicity of $hb \cup co$. Calibration confirms that RA admits 2+2W while SRA forbids it. Native `MFENCE` tests are excluded from RA/SRA runs.

CDSChecker required explicit C++11 compilation, non-PIE harnesses, and an increase to its pre-main bootstrap allocator on the current Linux toolchain. The patch, compiler options, revisions, bounds, and commands accompany the results. Tool errors and incomplete explorations fail the runner.

Remaining work includes full Dekker and repeated Peterson acquisitions, the turn-RMW repair in CDSChecker, and simplified RCU with an explicit reader-safety property. Queue slot reuse is a stretch goal. These extensions remain unverified. The final report will explain selected graphs and distinguish bounded evidence from any hand proof.

6. Included artifacts and references

The accompanying project contains `models/`, `litmus/`, `cdschecker_tests/`, `scripts/`, `results/`, and this report's LaTeX source. Run `python3 scripts/run_experiments.py --graphs --pdf` from the project root to reproduce the suite and rebuild this writeup.

[1] Alglave and Maranget. *herdtools7*, version 7.58, and herd7 documentation. Pinned source.

[2] Norris and Demsky. *CDSChecker: Checking Concurrent Data Structures Written with C/C++ Atomics*. OOPSLA 2013. Source and usage documentation.

[3] Lahav, Giannarakis, and Vafeiadis. *Taming Release-Acquire Consistency*. POPL 2016. Paper.